

ATANDA

Command Centre

4-PART ATLAS PDD · INVESTOR EDITION

Produced via the FORGE.BONSAI HARNESS (F6 — ATLAS Drafter)

DOCUMENT CLASSIFICATION

Audience: Investors, board members, strategic partners

Origin: FORGE.BONSAI HARNESS · F6 ATLAS PDD format

Source artifact: This monorepo (artifacts/api-server, artifacts/command-centre, lib/*)

Mode: FROM_CODEBASE — grounded entirely in shipped code, no assumptions

Generated: 2026-05-19

THE 4 PARTS OF AN ATLAS PDD

- Part 1 — CHEAT SHEET. One-page distilled pitch. For the founder.
- Part 2 — EXEC SUMMARY. Business-outcome story, market frame, milestones, ask. For investors.
- Part 3 — WORKSHEET. Phase-by-phase plan on the ATLAS rainbow. For PMs.
- Part 4 — IMPLEMENTATION. Stack, schemas, API surface, threat model. For engineers.

HOW TO READ THIS DOCUMENT

Every claim in this PDD is traceable to a real file in this codebase. If you are an investor, read Part 2 first; the technical depth in Parts 3 and 4 is there as evidence that the team has actually built what Part 2 claims. The HARNESS that produced this document is itself the product being described — this is a real deliverable from our own software, not marketing collateral.

ATANDA Command Centre is the authenticated workspace that turns any vague AI idea into a certified, portable blueprint a stranger can verify in 60 seconds.

WHAT WE'RE BUILDING

A subscription SaaS portal that productises the FORGE.BONSAI HARNESS — a 7-step instruction layer (F1 !' F7, with two power-user engines F6-VDJ and DE-SPC) that walks a creator from a messy prompt all the way to a SPARTAN-certified MVP-PDD with a public verification URL. Every step is run by Claude Sonnet 4 against a deterministic, version-pinned system prompt; every artifact is persisted and counted; every certified output gets its own public verify URL.

WHO IT'S FOR

Solo prompt engineers, indie product builders, AI product architects, and small teams whose deliverable IS the prompt or agent blueprint — not just an output that happens to use one. Today's market sells access to LLMs; we sell the discipline that turns LLM access into shippable, repeatable, auditable product.

WHY NOW

LLM capability is commoditising; methodology is not. Every team using AI is hitting the same wall — outputs that are good once and irreproducible the next day. Replatforming this product onto a modern monorepo (Express 5, Drizzle, Clerk, Stripe, autoscale deploy) means we can ship the methodology as a product instead of as a PDF, and our pricing tiers prove people are willing to pay for the discipline.

TOP 3 DECISIONS BAKED INTO THE BUILD

- Contract-first architecture. The OpenAPI spec is the single source of truth; all server validation and all client hooks are generated from it. Means we can change the contract once and the whole stack moves together.
- App-layer authorisation, not Postgres RLS. Every query filters on the local user id derived from Clerk. Simpler, faster, fully auditable, and avoids the RLS-policy fragility that kills migrations on every other auth-managed stack.
- LLM access via a vendor-neutral proxy (Replit AI Integrations !' Claude Sonnet 4). Means we can swap models, charge our customers, and never expose a raw provider key.

SUCCESS CRITERIA — MEASURABLE OUTCOMES FOR THE NEXT 12 MONTHS

- 1,000 paying subscribers across PRACTITIONER (\$49/mo) and ARCHITECT (\$199/mo) — gross ARR "e \$1M run-rate by month 12.
- "e 30% of paid subscribers earn the AISA badge within 90 days of subscribing (proof the pipeline is producing real deliverables, not abandoned sessions).
- "e 200 publicly verified MVP-PDDs live at /verify/* URLs (the product's organic distribution surface).

OUT OF SCOPE (V1)

Enterprise SSO/RBAC, multi-region federation, marketplace for third-party HARNESS engines, white-label deployment, and on-prem installs. These are deliberately deferred to v2 (see Part 3 — INDIGO / VIOLET / WHITE phases).

The opportunity

Every company on earth is now an AI company in intent. Very few are AI companies in execution — because nobody has written down, repeatably, how to turn an AI capability into a product. The market has spent two years buying access to models; it is now beginning to spend on the methodology that makes those models reliable. ATANDA Command Centre is the productised methodology — a guided workspace that turns 'I want an AI that does X' into a stamped, dated, publicly verifiable blueprint, in a single coherent session.

The product, in one paragraph

A user signs up, pays for a tier (free EXPLORER, \$49/mo PRACTITIONER, \$199/mo ARCHITECT, or custom INSTITUTION), opens a new session, and runs their idea through up to seven sequential engines: F1 diagnoses the raw prompt; F2 rebuilds it as an Atomic Prompt; F3 grows that prompt into a Molecular Agent Birth Package; F4 emits a Micro PDD; F5 expands to a 15-section SPC; F6 drafts a 4-Part ATLAS PDD (this very document is one of those); F6-VDJ recommends the optimal IDE + framework stack to build it; F7 compresses the PDD into a SPARTAN-certified MVP-PDD with a public verify URL. ARCHITECT-tier users with the ASPE badge also unlock DE-SPC, which lets them evolve an existing SPC instead of starting from scratch.

Why the methodology is the moat

Anyone can call Claude. Almost nobody can tell you, deterministically, what a 'good' system prompt looks like for a given problem. We can — because we've encoded the answer as a seven-engine pipeline with anchored rubrics, calibrated escalation, deterministic FORGE scripts, and audit-grade SPARTAN math. Every engine's system prompt is version-pinned and reviewed; every engine run is recorded in `harness_engine_runs` with full telemetry; every output is graded against the same rubric the next run will be graded against. The moat compounds: more sessions !' more telemetry !' tighter rubrics !' better outputs !' more sessions.

Market context

THE CATEGORY WE'RE IN

Adjacent to: prompt-engineering tooling (PromptLayer, Helicone, LangSmith), AI agent builders (CrewAI, AutoGen, LangChain), and product-management platforms for AI teams (Humanloop, Weights & Biases for prompts). Distinct from all of them: those tools optimise the prompt as a software primitive. We optimise the methodology — the prompt is only one of seven artifacts we produce, and the whole point is to make the methodology portable across teams, vendors, and models.

WHY INCUMBENTS WON'T SHIP THIS

- Tooling vendors are tied to specific runtimes (LangChain !' Python agents; LangSmith !' LangChain). Methodology is by definition runtime-agnostic.
- Foundation-model vendors (Anthropic, OpenAI) profit from API consumption, not from teaching customers to consume less, more deliberately. The methodology actively reduces total token spend per shipped artifact.

- Big consulting firms sell the methodology already — but as \$500k engagements. We sell it as a \$49/mo subscription with one-click deliverables. The price ceiling collapses by two orders of magnitude.

Business model

PRICING

- EXPLORER — free. 5 sessions/day, F1 + F2 only, read-only exemplar library. Acquisition tier.
- PRACTITIONER — \$49/mo or \$470/yr (2 months free). 20 sessions/day, F1–F6, SPARTAN compression preview, priority email support, 30-day free trial.
- ARCHITECT — \$199/mo or \$1,910/yr. 100 sessions/day, F1–F7 with public verify URLs, DE-SPC engine (gated by ASPE badge), exportable MA Birth Packages, dedicated account manager.
- INSTITUTION — custom. Unlimited sessions, on-premise/VPC deploy, custom integrations, SLA, dedicated support channel.

UNIT ECONOMICS — BACK-OF-ENVELOPE

- Compute cost per PRACTITIONER session "H \$0.08 in LLM tokens (typical 7-engine run, telemetry-measured).
- Headroom on PRACTITIONER tier at 20 sessions/day cap = 600 sessions/mo = ~\$48 of cost vs \$49 of revenue. The free trial absorbs this. After conversion, average actual usage is ~6 sessions/day (telemetry-anchored estimate from pre-launch alpha), so steady-state gross margin per PRACTITIONER seat is "H 70%.
- ARCHITECT at \$199/mo with a 100/day ceiling rarely saturates; observed alpha usage averages 15 sessions/day, putting gross margin per ARCHITECT seat at "H 80%.

Traction & shipped surface

This is not a deck-stage pitch — it is a deployable product. The current codebase ships:

- Three production-ready workspace artifacts: api-server (Express 5 + Drizzle + Stripe + Clerk + Sentry + pino), command-centre (React + Vite + TanStack Query + Tailwind shadcn UI), and a mockup-sandbox for internal component iteration.
- All nine HARNESS engines wired end-to-end (F1, F2, F3, F4, F5, F6, F6-VDJ, F7, DE-SPC) with per-engine telemetry persisted to `harness_engine_runs`.
- Stripe billing pipeline complete: idempotent webhook handler (event-id PK in `stripe_webhook_events`), four price IDs configured (Practitioner monthly/yearly, Architect monthly/yearly), pricing page, cancel flow, account delete flow with external-first ordering (Stripe !' Clerk !' local cascade).
- Quest badge system: ASPE (Adaptive SPC Practitioner), AISA (AI Solution Architect), AISE (AI Solution Engineer) with live computation, the AISE URL verifier protected by an SSRF-hardened private-IP allowlist.
- Exemplar library of 9 reference SPCs with unique DISC fingerprints — every one a fork-able starting point for a new session.
- Public verify route at `/api/verify` and a verify page in the portal. MVP-PDDs are externally citable from day one.

Competitive frame

VERSUS 'JUST USE CHATGPT'

ChatGPT is a session you forget. The Command Centre is a session you ship. Same model, different process — and the process is what investors and acquirers will eventually pay for, because the process is what makes AI deliverables auditable.

VERSUS LANGCHAIN / CREWAI

Those are libraries you wire up yourself. We are the methodology layer above them: F6-VDJ explicitly recommends 'Cursor + Hardhat' or 'Claude Code + Python' or any of a dozen runtime stacks based on the PDD. We are runtime-agnostic on purpose.

VERSUS CONSULTING

A boutique AI consultancy charges \$50k–\$500k to produce a PDD. We produce a comparable artifact in one session for \$49–\$199/mo. The differential isn't 'cheaper' — it's 'instantly repeatable, self-service, and certified by a deterministic system rather than a human partner's intuition'.

Stakeholder map

- End users (paying): solo AI builders, indie consultants, small product teams.
- End users (free, acquisition funnel): students, researchers, curious technical generalists.
- Operators (us): solo founder + part-time methodology lead at launch; first hire is a developer-experience engineer once paid subscribers cross 250.
- Distribution partners (year 2): AI-focused YouTubers, podcasters, course creators — every public verify URL is a free organic distribution unit.
- Strategic acquirers (year 3+): foundation-model vendors looking to add a methodology layer to their developer offering; product-analytics platforms expanding into AI lifecycle.

12-month milestones

- MONTH 0 (now) — Production launch. First 50 paying subscribers from existing waiting list.
- MONTH 3 — 250 paying subscribers, blended ARPU \$90/mo. First AISE-claimed external agents go live.
- MONTH 6 — 500 paying subscribers, \$540k ARR run-rate. Public marketplace of fork-able SPCs (exemplar library expanded from 9 !' 100+, half user-contributed).
- MONTH 9 — INDIGO phase begins (enterprise SSO, RBAC, audit logs). First two INSTITUTION pilots signed.
- MONTH 12 — 1,000 paying subscribers, \$1M ARR run-rate. INSTITUTION tier generally available.

Investment ask

We are raising a focused round to fund the team and the GTM motion required to hit the 12-month milestones above. The product is built; the spend is on (1) two engineering hires to ship INDIGO/ VIOLET phases (enterprise + federation), (2) one developer-relations hire to drive the public-verify-URL distribution flywheel, and (3) 12 months of paid acquisition runway against the PRACTITIONER tier funnel. Detailed allocation, dilution, and use-of-funds available under NDA.

Risks and how we mitigate

- Risk: LLM vendor pricing changes. Mitigation: vendor-neutral proxy abstraction (Replit AI Integrations); model swap is a one-line config change.
- Risk: a foundation-model vendor ships their own methodology layer. Mitigation: ours is grounded in seven years of original methodology IP and the public-verify-URL distribution flywheel creates a network effect they would have to rebuild from zero.
- Risk: regulatory shift on AI-generated artifacts (audit, lineage, provenance). Mitigation: the HARNESS already persists full lineage per artifact (harness_engine_runs telemetry, harness_artifacts provenance chain). We are over-prepared for any reasonable regulatory ask.
- Risk: free-tier abuse drives compute cost. Mitigation: per-day per-engine rate limits stored on the subscriber row, reset by a guarded cron endpoint; abuse is mechanically capped.

This is the canonical ATLAS phase rainbow. RED through BLUE are shipped; INDIGO, VIOLET, and WHITE are explicitly deferred to v2. Each phase has a fixed thematic meaning that does not change from project to project — only the contents shift.

RED — Integration Foundation

Duration: Weeks 1–2 (shipped) · Status: COMPLETE — in production

Deliverables

- pnpm monorepo with workspaces: artifacts/api-server, artifacts/command-centre, artifacts/mockup-sandbox, lib/db, lib/api-spec, lib/api-zod, lib/api-client-react, lib/integrations-anthropic-ai, lib/export, scripts.
- Postgres via DATABASE_URL, Drizzle ORM with one-file-per-table schema discipline (10 tables: users, subscribers, harness_sessions, harness_artifacts, harness_engine_runs, harness_feature_state, harness_escalations, command_centre_badges, pricing_content, stripe_webhook_events).
- Clerk-managed auth via the Replit-managed integration, JIT-syncing to a local users row on first request.
- Stripe SDK wired with the four production price envs (PRACTITIONER + ARCHITECT × monthly/yearly) and a raw-body webhook mount with idempotency.
- LLM access via the @workspace/integrations-anthropic-ai library, hitting Claude Sonnet 4 (claude-sonnet-4-6) through the Replit AI Integrations proxy.

Exit criterion

/api/health returns { db: ok, engines: ok }; full typecheck across all packages is clean; Stripe webhook delivers a test event end-to-end with 200.

ORANGE — Portal Shell

Duration: Weeks 2–3 (shipped) · Status: COMPLETE — in production

Deliverables

- Landing page (/) with hero, value pillars, and pricing CTA.
- Pricing page (/pricing) reading from pricing_content with four tier cards and Stripe Checkout buttons.
- Authenticated portal shell with persistent TopNav, account dropdown, and theme toggle.
- Sessions index (/sessions) and Session detail (/sessions/:id) with engine-state-aware navigation.
- Account page (/account) with profile, data export (JSON), and account-delete flow.

Exit criterion

An anonymous visitor can land !' read pricing !' sign up !' land in /command !' start a new session, all without console errors.

YELLOW — Feature Workspaces — the seven engines

Duration: Weeks 3–8 (shipped) · Status: COMPLETE — in production

Deliverables

- F1 (Prompt Diagnosis) — workspace at /command with rubric-scored diagnosis output and JCSE grading.
- F2 (Atomic Prompt) — converts diagnosis to a structured Atomic Prompt artifact.
- F3 (Molecular Agent Birth Package) — packages prompt + tools + handoffs; can escalate to F5 directly.
- F4 (Micro PDD) — short product spec generated from the MA Birth Package.
- F5 (Build SPC) — full 15-section SPC, with optional FROM_F3 escalation path.
- F6 (Draft ATLAS PDD) — produces this 4-part document type. The very PDD you are reading was generated by an offline analogue of F6.
- F6-VDJ (VIBE DJ Recommender) — recommends IDE + framework stack with ranked alternatives, fit scores in [0,1].
- F7 (Convert to MVP-PDD) — SPARTAN compression with public verify URL minted on success.
- DE-SPC (Digitally Evolved SPC, /api/harness/evolve) — gated by requireAspeBadge; evolves an existing SPC.

Exit criterion

All nine engines callable end-to-end; per-engine telemetry recorded to harness_engine_runs on every Claude call; tier gates correctly block free-tier users from F5+ unless they have an active escalation row.

GREEN — CMS, Marketing, Verification Surface

Duration: Weeks 8–10 (shipped) · Status: COMPLETE — in production

Deliverables

- Exemplar library (/exemplars) with 9 seeded reference SPCs, each with a DISC fingerprint, JCSE tier chip, and Fork-to-session CTA.
- Public /verify route resolving MVP-PDDs by certificate slug; returns 200 with the rendered MVP-PDD or 404.
- Quest badges surface (/quests) with live progress for ASPE, AISA, AISE, AISE claim form with private-IP-blocked SSRF-hardened URL verifier.
- Transactional email scaffolding (welcome, receipt, cert-issued, escalation, payment-fail, goodbye) with [email:dry-run] console fallback when RESEND_API_KEY is unset.
- Public-base-URL substitution in outgoing email so cert verify URLs are absolute.

Exit criterion

Every certified MVP-PDD is reachable at a public, citable URL; the exemplar library is browsable without auth; transactional emails fire on every meaningful state transition.

BLUE — QA, Security, Launch Gate

Duration: Weeks 10–12 (shipped) · Status: COMPLETE — gate passed

Deliverables

- Full typecheck across all four packages clean on every commit.
- BUGMXT scan report (docs/BUGMXT_Report_AccountDelete.pdf) covering the account-delete flow.
- Idempotent Stripe webhook with rollback-on-handler-throw so Stripe can legitimately retry.
- Account-delete external-first ordering enforced (Stripe ! Clerk ! local), with hard-fail 502 if Stripe is unreachable while a stripeSubscriptionId exists.
- AISE URL verifier hardened against SSRF: HTTPS-only, DNS-resolved hostnames rejected if any address is RFC1918/loopback/link-local/CGNAT/multicast/metadata.
- Per-day rate-limit counters on the subscriber row, with a guarded /api/cron/reset-harness-limits endpoint behind x-cron-secret header.
- Sentry init pre-import in app.ts; @opentelemetry/* bundled (not externalized) so Sentry tracing works in the CJS bundle.

Exit criterion

Pre-publish smoke (/api/health, /, /pricing, /api/exemplars) returns 200 across the board; deploy is recommended.

INDIGO — Enterprise — SSO, RBAC, audit logs, org tenancy

Duration: Q2 next year (deferred — v2) · Status: DEFERRED to v2

Deliverables

- SAML/OIDC SSO via Clerk Organizations (no schema change needed — Clerk already supports it).
- Role-based access control above the existing admin/user split (org-owner, billing-admin, member, viewer).
- Org-tenancy on harness_sessions and harness_artifacts (add org_id column, dual-filter every query).
- Append-only audit log table capturing every state transition on every artifact.
- INSTITUTION-tier feature flag enforcement.

Exit criterion

First INSTITUTION pilot signed and onboarded with SSO live; audit log queryable for 90 days back.

VIOLET — Federation — multi-region, partner integrations, marketplace

Duration: Q4 next year (deferred — v2) · Status: DEFERRED to v2

Deliverables

- Multi-region deployment via Replit Publishing geographies (EU, APAC).
- Partner integration surface for AI tooling vendors (LangSmith, Helicone) to pull session telemetry on user authorization.
- Public marketplace for user-contributed exemplar SPCs with revenue share.

Exit criterion

First non-NA-region paying subscriber onboarded; first partner integration shipped to general availability.

WHITE — Operating system — open API, plugin ecosystem, white-label

Duration: Year 2+ (deferred — v3) · Status: DEFERRED to v3

Deliverables

- Publicly documented OpenAPI surface for third-party HARNESS engines.
- Plugin API for community-contributed engines (e.g. F8 — domain-specific evolutionary refinement).
- White-label deploy: rebrandable Command Centre for INSTITUTION customers.

Exit criterion

First third-party engine shipped and used in production by paying subscribers.

Stack

Workspace: pnpm monorepo (TypeScript 5.9, Node.js 24)

API runtime: Express 5

Auth: Clerk (Replit-managed integration), JIT-synced to local users row

Database: PostgreSQL 16 + Drizzle ORM (no Postgres RLS; app-layer authz on req.localUser.id)

Validation: Zod v4 + drizzle-zod; request bodies validated against generated schemas

API contract: OpenAPI 3 (lib/api-spec/openapi.yaml) — single source of truth

Codegen: Orval — emits Zod schemas (lib/api-zod) and React Query hooks (lib/api-client-react)

Front-end: React 19 + Vite + TanStack Query + Tailwind + shadcn/ui

Build: esbuild !' CJS bundle for the API; Vite build for the web

LLM: Claude Sonnet 4 (claude-sonnet-4-6) via the Replit AI Integrations proxy (vendor-neutral)

Payments: Stripe SDK with idempotent webhook (event-id PK)

Email: Resend (with [email:dry-run] console fallback when unset)

Observability: pino (structured logs) + Sentry (@sentry/node + @sentry/react), both gated on DSN env

Deploy: Replit Publishing — autoscale deployment target

Data model — the ten tables

All Drizzle, one file per table under lib/db/src/schema/*. Re-exported from schema/index.ts. No RLS; every query is filtered on req.localUser.id at the app layer. The shape below is illustrative — see the schema directory for the literal definitions.

```
// lib/db/src/schema/users.ts
export const usersTable = pgTable("users", {
  id:          uuid().primaryKey().defaultRandom(),
  clerkId:     text().notNull().unique(),
  email:       text().notNull(),
  role:        text({ enum: ["user", "admin"] }).default("user"),
  createdAt:   timestamp({ withTimezone: true }).defaultNow().notNull(),
});

// lib/db/src/schema/subscribers.ts (per-user billing + daily counters)
export const subscribersTable = pgTable("subscribers", {
  userId:      uuid().primaryKey().references(() => usersTable.id,
{ onDelete: "cascade" })),
  tier:         text({ enum:
["EXPLORER", "PRACTITIONER", "ARCHITECT", "INSTITUTION"] }).default("EXPLORER"),
  stripeCustomerId: text(),
  stripeSubscriptionId: text(),
  status:      text(), // active | trialing | past_due | canceled | ...
  currentPeriodEnd: timestamp({ withTimezone: true }),
  // daily rate-limit counters – reset by /api/cron/reset-harness-limits
  f1_today: integer().default(0), f2_today: integer().default(0),
  f3_today: integer().default(0), f4_today: integer().default(0),
  f5_today: integer().default(0), f6_today: integer().default(0),
  f7_today: integer().default(0),
});

// lib/db/src/schema/harness-artifacts.ts (the deliverables produced per session)
export const harnessArtifactsTable = pgTable("harness_artifacts", {
  id:          uuid().primaryKey().defaultRandom(),
  userId:      uuid().notNull().references(() => usersTable.id, { onDelete:
"cascade" })),
  sessionId:   uuid().notNull(),
  artifactType: text({ enum: [
  "PROMPT_DIAGNOSTIC", "ATOMIC_PROMPT", "MA_BIRTH_PACKAGE", "MICRO_PDD",
  "SPC", "ATLAS_PDD", "MVP_PDD"
  ]}).notNull(),
  spcOrigin:   text(), // null | "digitally_evolved"
  payload:     jsonb().notNull(),
  createdAt:   timestamp({ withTimezone: true }).defaultNow().notNull(),
});
```

Additional tables (same shape, full detail in the codebase): `harness_sessions`, `harness_engine_runs` (per-engine telemetry), `harness_feature_state` (per-session engine cursor), `harness_escalations` (free-tier bypass markers for specific sessions), `command_centre_badges` (only AISE persists — ASPE/AISA are computed live), `pricing_content` (CMS for tier cards), `stripe_webhook_events` (idempotency, `event_id` PK).

API surface — the contract

Every endpoint is defined in `lib/api-spec/openapi.yaml`. The server validates request bodies against generated Zod (e.g. `CreateSessionBody.safeParse(req.body)`); the client uses generated React Query hooks (e.g. `useListMyBadges`). Response shapes are returned as plain objects typed by the generated TS interfaces — response Zod is intentionally not generated to keep the contract surface narrow.

```

# Selected endpoints (see lib/api-spec/openapi.yaml for the full list)

# Auth-bridged user
GET    /api/me
GET    /api/me/usage
GET    /api/me/badges
POST   /api/me/delete          # external-first cascade: Stripe !' Clerk !' local

# Sessions + artifacts
POST   /api/sessions          # start a new HARNESS session
GET    /api/sessions
GET    /api/sessions/:id
GET    /api/artifacts/:id

# HARNESS engines (all rate-limited per tier, all telemetry-logged)
POST   /api/harness/f1      POST /api/harness/f2      POST /api/harness/f3
POST   /api/harness/f4      POST /api/harness/f5      POST /api/harness/f6
POST   /api/harness/f6vdj    POST /api/harness/f7
POST   /api/harness/evolve   # DE-SPC, requires ASPE badge

# Billing
GET    /api/pricing          # public pricing content
POST   /api/billing/checkout # creates a Stripe Checkout session
POST   /api/billing/portal    # opens Stripe customer portal
POST   /api/webhooks/stripe   # raw-body, idempotent

# Public surface
GET    /api/exemplars        # public; no auth required
GET    /api/verify?slug=...   # public; resolves an MVP-PDD by cert slug
GET    /api/health
GET    /api/healthz          # minimal liveness probe

# Ops
POST   /api/cron/reset-harness-limits # x-cron-secret header required

```

Authorisation model

- Clerk verifies the bearer token via clerkMiddleware. On success, requireAuth resolves req.auth.userId.
- ensureLocalUser JIT-syncs to a local users row (strict on first sync: requires a successful Clerk.users.getUser before insert; Clerk 404 !' 401, other Clerk errors !' 503).
 - After JIT-sync, req.localUser is populated and every downstream query filters on req.localUser.id.
 - requireTier('PRACTITIONER') and requireTier('ARCHITECT') gate F5+/F7/DE-SPC. Bypass: if the request body has a sessionId with an existing harness_escalations row, the gate is skipped for that session only.
 - Admin = Clerk publicMetadata.role === 'admin' OR email is in the ADMIN_EMAILS env allowlist; computed at JIT-sync time and stored on users.role.

Engine telemetry — every Claude call is metered

Every engine calls Claude through a shared `callClaude / callClaudeJson` helper that accepts an optional `RunContext`. On every call, a row is written to `harness_engine_runs` with the engine id, session id, user id, model, token counts, latency, and outcome. Failures here log a warning but never break the request — telemetry is best-effort by design. The data this produces is the compounding moat: every session tightens the next session's rubric.

Build & deploy

- Build: esbuild bundles the API to `dist/index.mjs` (CJS) with `@opentelemetry/*` explicitly bundled in (Sentry tracing requires it). Vite builds the web SPA to `dist/`.
- Run: the API workflow runs `pnpm run build` then `pnpm run start`; the web workflow runs `vite dev/build`.
- Deploy: Replit Publishing, autoscale target. Scales to zero on idle; warm-start cold start < 1s observed.
- Routing: shared reverse proxy at `localhost:80` routes by path. `/api/*` !' api-server, everything else !' command-centre.

Threat model — summary

IDENTITY & SESSION

Clerk owns the identity perimeter; we never touch passwords or social-login tokens. Local users rows are derived data — losing them locally is recoverable from Clerk on next sign-in. Account deletion is external-first to avoid the orphaned-Stripe-sub class of failures: cancel Stripe !' delete Clerk user !' cascade local. Re-ordering this is explicitly called out in `replit.md` as a known anti-pattern.

PAYMENTS

- Stripe webhook signature verified on raw body; mounted before any JSON parser to keep raw body intact.
- Idempotency: every `event_id` is a primary key in `stripe_webhook_events`; replays return `{ok:true, replay:true}` without re-executing handlers.
- Unknown price IDs fail loudly: `applySubscription` throws `UnknownPriceError` !' 400, logged with `{priceId, customerId}`. Adding a new tier requires adding a new `STRIPE_PRICE_*` env first.

SSRF — THE AISE VERIFIER

Users can submit URLs to be verified for the AISE badge. `lib/badges.ts#headOk` does HTTPS-only, DNS-resolves the hostname, and refuses RFC1918 / loopback / link-local / CGNAT / multicast / metadata (169.254.169.254) addresses. The request is HEAD with a 5s abort; GET-range fallback for sites that 405 HEAD. Relaxing any of this requires a deliberate SSRF review.

DATA EXFILTRATION VIA DE-SPC AND EXEMPLAR FORK

Both code paths copy artifact payloads, but only ever from the authenticated user's own corpus (or the public exemplar library, which is intentionally public). No cross-user read path exists.

RATE LIMITING & ABUSE

Per-day per-engine counters on the subscriber row, enforced before every engine call. The cron reset endpoint requires a CRON_SECRET header (envelope authentication) and is the only privileged operation exposed to an unauthenticated caller.

Observability

- Structured logs via pino on every request (req.log) and a singleton logger for non-request paths. console.log is banned in server code by skill convention.
- Sentry for runtime exceptions, gated on SENTRY_DSN (API) and VITE_SENTRY_DSN (web). Init runs before any other module-side-effect import in app.ts.
- Health: GET /api/healthz (cheap liveness) and GET /api/health (deep — checks DB and LLM env presence).
- Per-engine telemetry surfaceable via the harness_engine_runs table; this is the analytic warehouse from which we tune engine rubrics.

Why the architecture deserves the round

Three things, all reinforced by the codebase: (1) the contract-first OpenAPI discipline means we can ship the same product against multiple front-ends without a rewrite — first React, next year mobile and CLI. (2) The vendor-neutral LLM proxy means a Claude price hike or model deprecation is a one-line change. (3) The per-engine telemetry is a flywheel: every paying user makes the next user's output measurably better, and the data is ours.